

IP3

Requirements:

- submit a project that creates module(s) to solve a chosen task in a chosen domain
(*eg. write an application that prints a list of specific students sorted by their grades*)
- create a container to process class hierarchies and demonstrate their polymorphic behaviour
- understand and implement several design patterns
https://en.wikipedia.org/wiki/Software_design_pattern (see below)
- implement the Strategy design pattern https://en.wikipedia.org/wiki/Strategy_pattern allowing to change parts of their container – create a hierarchy of multiple objects which are interchangeable
(*eg. allow grades to be computed and students to be sorted differently*)
- implement the Prototype design pattern https://en.wikipedia.org/wiki/Prototype_pattern allowing to properly copy objects in the hierarchy without knowing their exact type
- can only be started once IP2 was approved and has been uploaded to Moodle
- your choice must be explicitly approved, must be unique so that no two students share the same problem domain
- you are allowed to take implementation online or use AI, but you should credit it properly

Inheritance:

- create a class hierarchy as follows: have a base class, have (at least) two (specific) classes that are descendants of the base class and design the third class (ok to just plan and document if you are more experienced; otherwise create it also to make sure that it is actually possible and will work as you intend)
(*eg. Students can be from a Bachelor or from a Master studies as well as PhD students*)
- the hierarchy should have a single header file (.h), each class to be put in a separate implementation file (.cpp) two of which should contain implementations and the third should contain a description of the third class (implementation will be someone else's job, thus write the description that would be useful)
- allow for polymorphic copying (Prototype pattern) which is to be used when doing a deep copy for the container (see below)
- have at least one (more) virtual method intended to demonstrate polymorphism; for this purpose, in a file "demo.cpp" write code and provide a line where polymorphism occurs; right above this line write a single-line comment with (only) the word "polymorphism"
(*this is what you create the hierarchy for – so what do your objects do? What is it that (almost) all objects can do, but each of them do it (slightly) differently? Answer this question when picking a project, before writing any code, and explicitly state it when requesting approval from me*)
- have at least three different implementations of this virtual method (you have 4 classes, including the one designed but not implemented; however, the base class can be abstract), one of them must forbid calling the method (done by throwing NotImplementedException)
- have at least one method which is present in a child class, but not in the base class, and in the file "demo.cpp" access it through the pointer of the base class by using dynamic_cast operator
(*eg. PhD students can have research articles published while the other students do not*)
- provide other methods allowing to work with the hierarchy and do the proper demonstration (eg. provide operator< if you are doing sorting)
(*eg. compare students by their name and then their surname*)

Container:

- create a class designed to contain (pointers to) instances of the base class from the above-mentioned hierarchy (ok to implement using a STL container)

- write your own iterator (implementation can delegate work to iterator of a chosen STL container or not) as the inner class, name it according to its type (check 5 types of iterators used by STL) and implement all the operations required by the type
- provide CRUD operations for the container as well as `begin()` and `end()` methods returning appropriate iterators
- in the file “demo.cpp” demonstrate the usage of your iterator, calling at least one function from `<algorithm>` (eg. `sort(myContainer.begin(), myContainer.end());`; mark the line with a single-line comment above with (only) the word “iterator”
- all fields of the container must be in dynamic memory (use `pimpl` to hide the implementation in the .cpp file) and container must provide deep copy (use polymorphic copying to ensure that object types are copied properly) – create at least one copy of your container in “demo.cpp” and mark the line with a single-line comment above with (only) the words “deep copy”
- container should implement the Strategy design pattern; provide the method which would delegate the functionality to a hierarchy of objects for different strategies, and provide an additional method to set the current strategy to be used; if the strategy is not set, but an attempt to use it is made, the method should throw `StrategyNotSet` exception; have a line in “demo.cpp” where you set a strategy for a container, and mark the line with a single-line comment above with (only) the word “strategy”
(answer in advance when making a proposal: what is the action that container can do differently? What is the action that can be performed using different strategies to be specified, set and used later?)
- create at least one method taking a callback function or object as a parameter and provide at least two versions to be passed (can delegate it to `<algorithms>`), so that the decision is dependent not on the type of the object in the hierarchy, but on the arbitrary object passed as a parameter – do a demonstration in “demo.cpp” and mark the line with a single-line comment above with the word “callback”
(what is the action that container can perform which would get a functional object as a parameter to specify part of its behaviour? Also to be answered when picking a project)
- have at least one method in the container that passes through all the objects in the container and calls their virtual method to obtain some desired result, call it in the file “demo.cpp”

The third class:

- must be designed to be derived from the base class
- must NOT be implemented; instead, the description must be provided without the code, in a separate .cpp file where the code should be written; to implement it will be the job of your colleagues who get your project after it has been submitted
- must be used in tests (create the file “test.cpp” where test your container and the hierarchy); while the class is neither declared nor implemented, it must be used in tests which would show how the yet-to-be-created class should be used

Files:

- .h and .cpp for your class hierarchy (including declarations and definitions for all except that one “third” class, for which provide a separate .cpp file without any code, with a detailed description of how the class should be written)
- .h and .cpp for your container as well as .h and .cpp for the strategies you use in the container
- demo.cpp where you demonstrate that your project handles all the requirements – must have five single-line comments with words “callback”, “deep copy”, “iterator”, “polymorphism”, and “strategy” (in any order)
- test.cpp where you test your classes and (most importantly) provide test for the “third” yet-to-be-implemented class, showcasing how it should be used and what would successful implementation would look like
- have makefile.txt which allows building from command line: a) to build the module.o from .h and .cpp) to build demo c) to run it d) to build test e) to run it f) to clean everything and g) to rebuild from scratch
- have documentation either in .h (recommended) or in a separate file

- upload all files to a single folder in Git without any extra files or folders (eg. from IDE)

Summary:

- understand the essence and purpose behind these requirements: you are asked to build a solution which has fixed and changeable parts, and you must provide means to change them through adding new classes and without changing the existing code
- virtual method in class hierarchy must specify something that the hierarchy itself can change, i.e. derived classes specify what new behaviour is; you change behaviour by creating a new derived class
- strategy in the container sets what the container (and not the hierarchy) can do differently; you change behaviour by creating a new strategy and setting it in a container
- callback objects are designed to be used as parameters, allowing a user to control and change what the particular method does; you change behaviour by creating a new object to be used for callback
- these three (changeable) things must be named in a proposal for me to approve your idea
- additionally, you are asked to implement polymorphic copying as well as your own iterator

Example:

- create a University class that contain various Students – BSc, MSc, and PhD students (the latter is described but not provided and thus will be implemented later) – implementing essential functionality; for all students grades are available, and for PhD students NotImplementedException is thrown
- for the hierarchy polymorphic copying is provided as well as deep copy for the University class
- create an iterator allowing to go through all or some students (use the Prototype pattern to define objects of interest) in the University
- create a method that computes the average grade of students; similarly, create methods that compute the maximal or the minimal grades; implement them in separate classes thus implementing Strategy pattern while the compute() method in the container allows to compute one thing at a time
- create a sort() method that takes callback object as a parameter, specifying the sorting order
- important: you should NOT copy the example as it is more complicated than required of you (read the requirements) and also is considered as taken; treat it merely as a tool to understand what requirements ask you to do (i.e. you are asked to build a module and NOT the program; before attempting the task, for this example, think what would your own naïve solution would look like, and then check what is being proposed, to allow various computations as well as various sorting, to be easy to change and including ways are not yet known)

Timeframe:

- to be submitted during April